return the object requested along with information about *where* to go next and *links* to perform related operations. The beauty of this feature is that the links are only shown when it makes sense to follow them, providing information about *when* to go.

This hypermedia aspect is also called *HATEOAS* (Hypermedia as the Engine of Application State), and it represents the highest level of API maturity according to Richardson's Maturity Model. Spring provides the Spring HATEOAS project to add hypermedia support to a REST API. I will not use it in this book, but I encourage you to check the project's online documentation at https://spring.io/projects/spring-hateoas.

These considerations conclude our discussion of building a RESTful application with Spring. In the next section you'll see how to write automated tests to verify the behavior of your application.

3.4 Testing a RESTful application with Spring

Automated tests are paramount to producing high-quality software. One of the goals for adopting a cloud native approach is speed. It's impossible to move quickly if the code is not adequately tested in an automated fashion, let alone to implement a continuous delivery process.

As a developer, you'll usually implement a feature, deliver it, and then move on to a new one, possibly refactoring the existing code. Refactoring code is risky, since you might break some existing functionality. Automated tests reduce the risk and encourage refactoring, because you know that a test will fail, should you break something. You'll probably also want to reduce the feedback cycle so you'll know if you made any mistakes as soon as possible.